

Aufgabe 1: Ankreuzfragen (24 Punkte)

1) Einfachauswahlfragen (16 Punkte)

Bei den Einfachauswahlfragen in dieser Aufgabe ist jeweils nur **eine** richtige Antwort eindeutig anzukreuzen. Auf die richtige Antwort gibt es die angegebene Punktzahl.

Wollen Sie eine Antwort korrigieren, streichen Sie bitte die falsche Antwort mit drei waagrechten Strichen durch (~~☒~~) und kreuzen die richtige an.

Lesen Sie die Frage genau, bevor Sie antworten.

a) Gegeben seien die folgenden Präprozessor-Makros:

```
#define ADD(a, b) a + b
```

```
#define DIV(a, b) a / b
```

Was ist das Ergebnis des folgenden Ausdrucks?

```
3 * DIV(ADD(4, 8), 2)
```

- ☐ 10
☐ 18
☐ 16
☐ 24

b) Ein Programm will die drei Zeichenketten

```
char a[] = "dire";
```

```
char b[] = "cto";
```

```
char c[] = "ry";
```

mit der Funktion `sprintf(3)` wie folgt in den Puffer `buffer` speichern:

```
sprintf(buffer, "%s/%s/%s", a, b, c);
```

Mit welcher Länge (in Bytes) muss der Puffer `buffer` mindestens angelegt werden, damit kein Überlauf entstehen kann?

- ☐ 12
☐ 10
☐ 11
☐ 13

c) Was versteht man unter virtuellem Speicher?

- ☐ Adressierbarer Speicher in dem sich keine Daten speichern lassen, weil er physikalisch nicht vorhanden ist.
- ☐ Virtueller Speicher kann größer sein als der physikalisch vorhandene Arbeitsspeicher. Gerade nicht benötigte Speicherbereiche können auf den Hintergrundspeicher ausgelagert werden.
- ☐ Virtueller Speicher ist in unbegrenzter Menge vorhanden.
- ☐ Speicher, der nur im Betriebssystem sichtbar ist, jedoch nicht für einen Anwendungsprozess.

d) Welche Aussage zu Prozesszuständen ist richtig?

- ☐ Der Zustand blockiert (*blocked*) zeigt an, dass der Prozess sich in einem Dead-lock befindet.
- ☐ Während ein Prozess darauf wartet, dass er vom Scheduler eingelastet wird, befindet er sich im Zustand blockiert (*blocked*).
- ☐ Der Zustand bereit (*ready*) zeigt an, dass ein Prozess bereit ist, Daten von einem I/O-Gerät zu empfangen.
- ☐ Ein Prozess kann nicht direkt vom Zustand bereit (*ready*) in den Zustand blockiert (*blocked*) übergehen.

e) Welche der Aussage zum Thema Threads ist richtig?

- ☐ Zu jedem Thread (*Light-weight Process*) gehört ein eigener Adressraum.
- ☐ Zu jedem User-Level Thread (*Feather-weight Process*) gehört ein eigener Adressraum.
- ☐ Bei Threads (*Light-weight Processes*) ist die Schedulingstrategie durch das Betriebssystem vorgegeben.
- ☐ Bei User-Level Threads (*Feather-weight Processes*) ist die Schedulingstrategie durch das Betriebssystem vorgegeben.

f) Welche Aussage zu Multilevel-Feedback-Queues ist richtig?

- ☐ Multilevel-Feedback-Queues bevorzugen CPU-intensive Prozesse gegenüber I/O-intensiven Prozessen.
- ☐ Laufende (*running*) Prozesse nutzen die Rückmeldung (*Feedback*) des Kernels, um ihr Laufzeitverhalten an die aktuelle Systemauslastung anzupassen.
- ☐ Multilevel-Feedback-Queues bestehen aus mehreren Round Robin Queues mit unterschiedlichen Zeitquanten, um die Prioritäten von Prozessen besser abzubilden.
- ☐ Wenn ein Prozess längere Zeit nicht ausgeführt wurde, kann der Scheduler den Prozess wieder in ein Level mit höherer Priorität verschieben.

g) Welche Seitennummer (*page number*) und welcher Versatz (*offset*) gehören bei einstufiger Seitennummerierung und einer Seitengröße von 1024 Bytes zu folgender logischen Adresse: 0xc01a ?

2 Punkte

- ☐ Seitennummer 0xc, Versatz 0x1a
- ☐ Seitennummer 0xc0, Versatz 0x1a
- ☐ Seitennummer 0xc01, Versatz 0xa
- ☐ Seitennummer 0x30, Versatz 0x1a

h) Welche Aussage zum Thema Wettlaufbedingungen ist richtig?

2 Punkte

- ☐ Auf Prozessoren mit einem einzigen Kern können keine Wettlaufsituationen (*Race Conditions*) eintreten.
- ☐ Die Reihenfolge, in der Locks freigegeben (*release*) werden, hat keinen Einfluss darauf, ob ein Deadlock auftritt.
- ☐ Die Reihenfolge, in der Locks erworben (*acquire*) werden, hat keinen Einfluss darauf, ob ein Deadlock auftritt.
- ☐ Der gleichzeitige, unsynchronisierte Zugriff auf Daten kann zu einem Deadlock führen.

2) Mehrfachauswahlfragen (8 Punkte)

Bei den Mehrfachauswahlfragen in dieser Aufgabe sind jeweils m Aussagen angegeben, davon sind n ($0 \leq n \leq m$) Aussagen richtig. Kreuzen Sie alle richtigen Aussagen an.

Jede korrekte Antwort in einer Teilaufgabe gibt einen Punkt, jede falsche Antwort einen Minuspunkt. Eine Teilaufgabe wird minimal mit 0 Punkten gewertet, d. h. falsche Antworten wirken sich nicht auf andere Teilaufgaben aus.

Wollen Sie eine falsch angekreuzte Antwort korrigieren, streichen Sie bitte das Kreuz mit drei waagrechten Strichen durch (~~☒~~).

Lesen Sie die Frage genau, bevor Sie antworten.

a) Konzeptionell ist der Speicher eines UNIX-Prozesses in Text-, Daten- und Stack-Segment untergliedert. Welche der folgenden Aussagen treffen zu?

4 Punkte

- ☐ Variablen der Speicherklasse *static* liegen im Daten-Segment.
- ☐ Lokale *automatic* Variablen einer Funktion werden im Stack-Segment abgelegt.
- ☐ Variablen der Speicherklasse *automatic* werden durch den Übersetzer mit dem Wert 0 initialisiert.
- ☐ Zeigervariablen können auf Daten aus allen Segmenten verweisen.
- ☐ Vor Ausführung einer Funktion wird das Daten-Segment vergrößert, um den Speicher für lokale Variablen zu reservieren.
- ☐ Lokale Variablen der Speicherklasse *static* werden beim Betreten der zugehörigen Funktion neu initialisiert.
- ☐ Dynamisch allozierte Zeichenketten werden in das Text-Segment gelegt.
- ☐ Das Text-Segment enthält sowohl den Programmcode als auch konstante Zeichenketten.

b) Welche der folgenden Aussagen zum Thema Threads und Prozesse sind richtig?

4 Punkte

☐ Die Einplanung und Einlastung von User-level Threads (*Feather-weight Processes*) findet ohne Wissen des Betriebssystems in der Anwendung statt.

☐ Zur Umschaltung von User-level Threads (*Feather-weight Processes*) ist ein Adressraumwechsel erforderlich.

☐ User-level Threads (*Feather-weight Processes*) blockieren sich bei blockierenden Systemaufrufen gegenseitig.

☐ Threads (*Light-weight Processes*) können Multiprozessoren ausnutzen.

☐ Zu jedem Thread (*Light-weight Process*) gehört ein eigener isolierter Adressraum.

☐ Die Umschaltung von Threads (*Light-weight Processes*) muss im Systemkern erfolgen.

☐ Threads (*Light-weight Processes*) teilen sich den kompletten Adressraum und verwenden daher den selben Stack.

☐ Die Veränderung von Variablen und Datenstrukturen in einem mittels `fork()` erzeugten Kindprozess beeinflusst auch die Datenstrukturen im Elternprozess.

Klausur Betriebssysteme

September 2024

Aufgabe 2: pronto (Parallel-Rasender Online-Nachrichten Transport-Organizer) (44 Punkte)

Sie dürfen diese Seite zur besseren Übersicht bei der Programmierung heraustrennen!

Schreiben Sie ein POSIX-1.2008 und C11-konformes Programm `pronto`, welches personalisierte E-Mails an mehrere Empfänger versendet. Der Versand soll hierbei parallelisiert in mehreren Prozessen erfolgen. Die Empfänger sind in der Datei `recipients.txt` aufgelistet. Jede Zeile enthält den Namen und die E-Mail-Adresse eines Empfängers. Name und E-Mail-Adresse werden durch ein Semikolon getrennt.

Eine Vorlage für die zu versendende Mail befindet sich in der Datei `template.txt`. Der Text der Vorlage muss noch für die jeweiligen Empfänger angepasst werden, indem der Text `<NAME>` durch den Namen des Empfängers ersetzt wird. Das Anpassen der Vorlage, sowie das Versenden der Mail, soll hierbei für jeden Empfänger in einem eigenen Prozess erfolgen. Gehen Sie davon aus, dass sich die Dateien `recipients.txt` und `template.txt` im aktuellen Arbeitsverzeichnis befinden.

Aufbau der Datei <code>recipients.txt</code>	Aufbau der Datei <code>template.txt</code>
Manuel Vögele;manuel.voegele@rub.de Henriette Herzog;henriette.herzog@rub.de Timo Hönig;timo.hoenig@rub.de	Hallo <code><NAME></code> , Sie haben gewonnen! Senden Sie all Ihr Geld in Bitcoin an die folgende Adresse, um Ihren Gewinn einzulösen...

Das Programm soll folgendermaßen strukturiert sein:

- Funktion `int main(void)`:
Die Dateien `template.txt` und `recipients.txt` werden zunächst mittels der Funktion `read_from_file` eingelesen. Anschließend wird über die Empfänger iteriert. Dabei wird jeder Empfänger in Name und E-Mail-Adresse aufgespalten. Dann werden Name und E-Mail gemeinsam mit der Vorlage an die Funktion `send_email` übergeben. Sobald sich alle Prozesse beendet haben, räumt die Funktion `main` den verbleibenden Speicher auf und beendet sich.
- Funktion `pid_t send_email(char *template, char *name, char *email)`:
Es wird zunächst ein neuer Prozess erzeugt, der die weitere Verarbeitung übernimmt. Tritt bei der Prozesserzeugung ein Fehler auf, soll das Programm nicht beendet, sondern die Abarbeitung mit den übrigen Mailadressen fortgesetzt werden. Innerhalb der Vorlage (`template`) soll nach dem Text `<NAME>` gesucht werden, der durch den Namen des Empfängers ersetzt wird. Anschließend wird das Programm `sendmail` durch eine Funktion der `exec`-Familie aufgerufen, um die Mail zu versenden. Im Elternprozess gibt `send_email` die Prozess-ID des Kindprozesses zurück.
- Funktion `char *read_from_file(const char *filename)`:
Die Datei, deren Name über den Parameter `filename` benannt wurde, wird geöffnet. Anschließend wird der Dateinhalt vollständig in einen String gelesen. Der hierfür benötigte Speicher wird von `read_from_file` allokiert und bei Bedarf dynamisch vergrößert. Beim Beenden der Funktion wird der eingelesene String zurückgegeben. Das Freigeben des Strings wird dem Aufrufer überlassen.

Hinweise:

- Sie dürfen davon ausgehen, dass die Datei `recipients.txt` immer korrekt formatiert ist.
- Sie dürfen davon ausgehen, das `<NAME>` in der Datei `template.txt` immer exakt einmal vorkommt.
- Der Betreff der E-Mail darf einen beliebigen Text enthalten.
- Eine Beschreibung des Kommandos `sendmail` finden Sie in den angehängten Manpages.
- Es ist **keine** Fehlerbehandlung/`fflush()` für Ausgaben auf `stdout` und `stderr` nötig. Achten Sie ansonsten auf korrekte und vollständige Fehlerbehandlung.

- 7 von 17 -

- 8 von 17 -

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>

static void die(char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

// Makros, Funktionsdeklarationen, globale Variablen

// Funktion main

// Empfänger und Vorlage laden

// Über Empfänger iterieren

// Zeile in Name und E-Mail aufspalten
```

```
// Mail versenden

// Warten auf Prozesse und Aufräumen

// Ende Funktion main
```

```
// Funktion send_email
```

```
// Neuen Prozess erstellen & Vorlage anpassen
```

1

7

```
// Mail versenden
```

7

```
// Ende Funktion send_email
```

S:

```
// Funktion read_from_file
```

10

```
// Datei öffnen
```

```
// Dateiinhalt in String lesen
```

10

11

```
// Aufräumen
```

10

```
// Ende Funktion read_from_file
```

R:

Vervollständigen Sie das folgende Makefile. Es soll ein Target `pronto` unterstützt werden, welches das Programm `pronto` baut. Greifen Sie dabei stets auf Zwischenprodukte (z.B. `pronto.o`) zurück.

Nutzen Sie dabei die Variablen CC und CFLAGS konventionskonform. Achten Sie darauf, dass das Makefile ohne eingebaute Variablen und Regeln (Aufruf von `make -Rr`) funktioniert!

.PHONY: all clean

CC=gcc

```
CFLAGS=-std=c11 -pedantic -Wall -D_XOPEN_SOURCE=700
```

all: pronto

clean:

```
rm -f pronto pronto.o
```

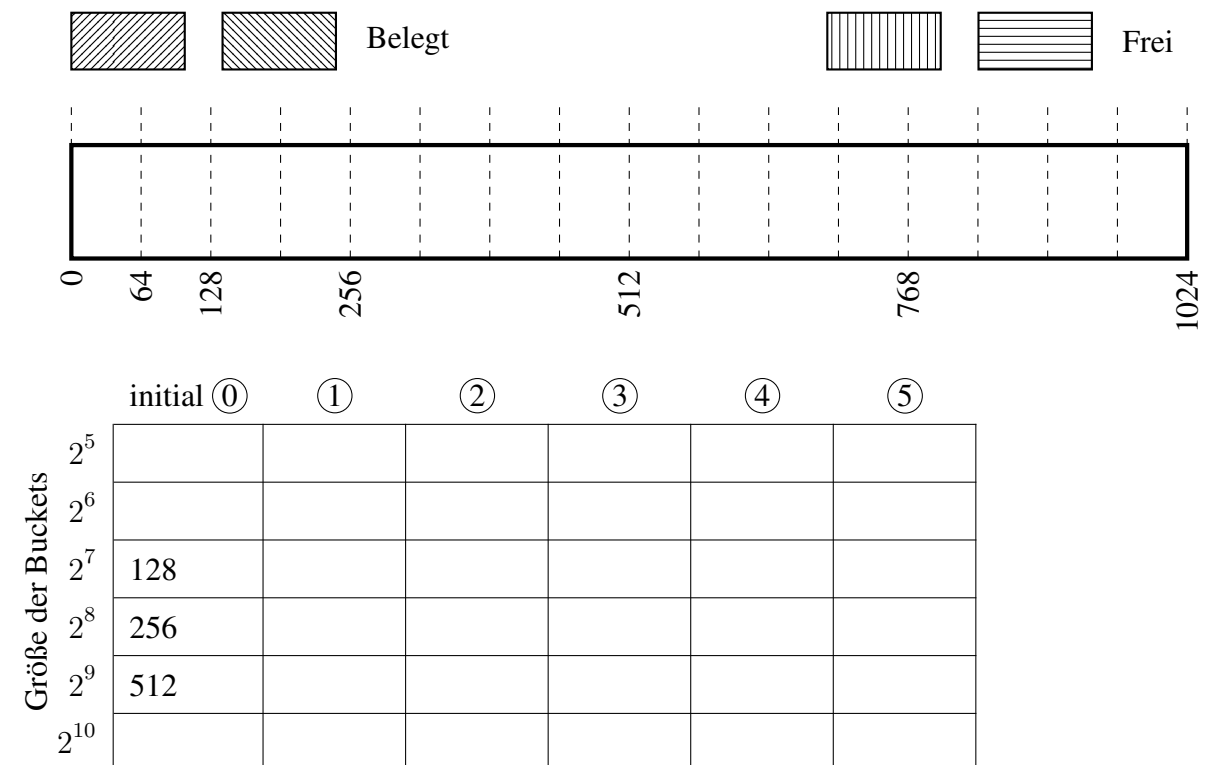
[illegible]

Aufgabe 3: Adressräume & Freispeicherverwaltung (12 Punkte)

1) Ein in der Praxis häufig eingesetztes Verfahren zur Verwaltung von freiem Speicher ist das *Buddy*-Verfahren. Nehmen Sie einen Speicher von 1024 Bytes an und gehen Sie davon aus, dass die Freispeicher-Verwaltungsstrukturen separat liegen. Initial ist bereits ein Datenblock der Größe 128 Bytes vergeben worden. Ein Programm führt nacheinander die im folgenden Bild angegebenen Anweisungen aus. (8 Punkte)

- ① p0 = malloc(100); // 0 (initial vergebener Block)
- ① p1 = malloc(22);
- ② p2 = malloc(128);
- ③ free(p0);
- ④ p3 = malloc(386);
- ⑤ free(p1);

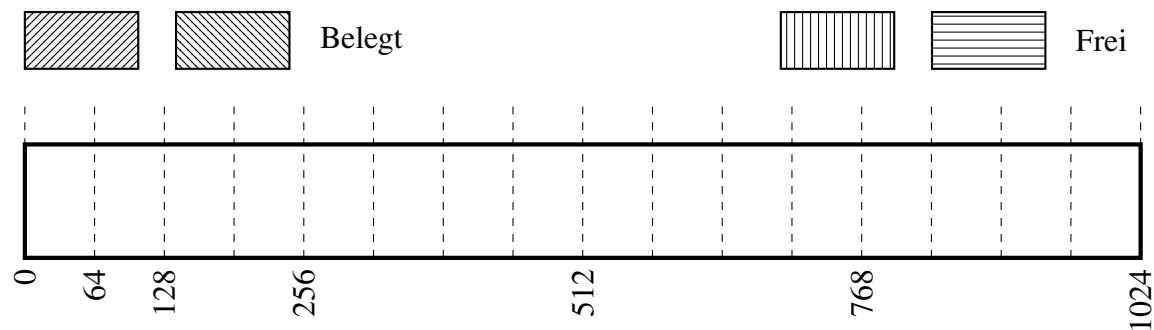
Tragen Sie hinter den obigen Anweisungen jeweils ein, welches Ergebnis die `malloc()` - Aufrufe zurückliefern. Skizzieren Sie in der folgenden Grafik, wie der Speicher **nach Schritt ③** aussieht, und tragen Sie in der Tabelle den aktuellen Zustand der Bucketliste (*left-over holes*) nach **jedem** Schritt ein. Für Löcher gleicher Größe schreiben Sie die Adressen einfach nebeneinander in die Tabellenzeile (es ist nicht notwendig, verkettete Buddys wie in der Vorlesung beschrieben einzutragen).



Hinweis: $2^5 = 32, 2^6 = 64, 2^7 = 128, 2^8 = 256, 2^9 = 512, 2^{10} = 1024$

[illegible]

Sie dürfen diese Grafik nutzen, falls Sie sich beim Ausfüllen der Grafik in 3.1. verzeichnet haben. Markieren Sie eindeutig, welche der Grafiken zur Bewertung herangezogen werden soll!



In einem System mit einem Prozessorkern (*single-processor system*) sollen fünf rechenintensive Prozesse mit den folgenden Zeiten ausgeführt werden:

<i>Prozess</i>	<i>arrival time</i>	<i>service time</i>
A	4	6
B	0	6
C	9	2
D	7	1
E	1	3

Zeichnen Sie in dem untenstehenden Diagramm für die gegebene Scheduling-Strategie ein, welcher Prozess zum jeweiligen Zeitpunkt aktiv (*running*) ist. Die Ready Queue kann als Gedankenstütze zur leichteren Bearbeitung der Aufgabe genutzt werden, wird jedoch nicht bepunktet.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
A										█	█	█	█	█	█	█					
B	█	█	█	█	█	█	█	█	█	█	█	█	█	█	█	█	█	█	█	█	█
C																	█	█	█	█	█
D	█	█	█	█	█	█	█	█	█	█	█	█	█	█	█	█	█	█	█	█	█
E							█	█	█	█	█	█	█	█	█	█					

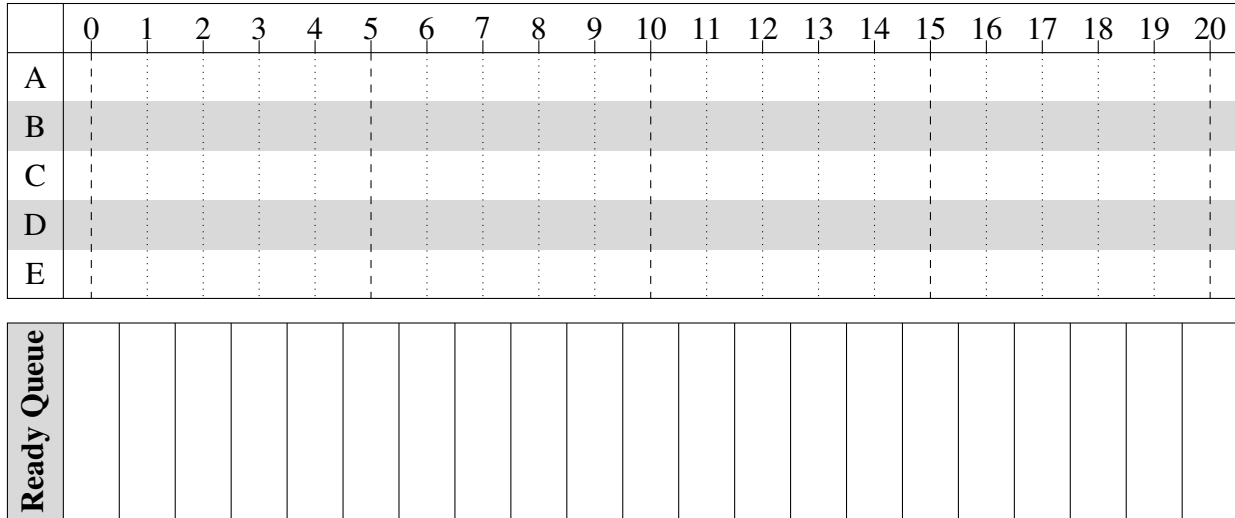
Ready Queue	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
	B	E	E	E	E A	E A	E A	A D	A D	A D C	D C	D C	D C	D C	D C	D C	C	∅	∅	∅	∅

[illegible]

Zur leichteren Bearbeitung wird hier die Prozesstabelle der vorherigen Seite noch einmal wiederholt:

Prozess	arrival time	service time
A	4	6
B	0	6
C	9	2
D	7	1
E	1	3

2) **Round Robin (RR)** mit Zeitquantum $q = 3$ (5 Punkte)



Ersatzgrafik für Aufgabe 4.

Sie dürfen diese Grafik nutzen, falls Sie sich beim Ausfüllen der Grafik in 4.1. verzeichnet haben. Markieren Sie eindeutig, für welche Teilaufgabe Sie diese Grafik verwendet haben und welche der Grafiken zur Bewertung herangezogen werden soll!

